FIN Planner

Operator Manual

Application version: 1.37.1

Database version: 2.23

Generated: 2026-10-28 12:51:05 UTC

FIN Planner — single-owner, self-hosted personal finance.

About this manual

A task-oriented reference for the owner of a single-user, self-hosted FIN Planner installation. Every procedure in this manual is supported by FIN Planner v1.37.1 (database v2.23) as distributed in this repository. The CLI commands, API actions, and on-screen dialogs referenced here are the only ones shipped by the current source.

This manual does **not** describe private deployment details (hostnames, institution names, or operator-specific paths) and does not invent procedures that the product does not support. Sections marked *Gap* list topics where the backlog item calls for documentation but the current source does not provide a procedure; those gaps are listed again at the end of this manual.

Canonical terminology

FIN Planner uses these user-facing terms throughout the application and therefore throughout this manual.

Term	Meaning
Account	A single real bank/payment position with one signed Balance. Negative, positive, and zero are all valid; Balance can also be *Unknown*. There is no Asset/Liability classification.
Envelope	A persistent named bucket of money for one purpose (Daily spend, Reserved, Savings).
Month	A calendar month plan (year, month number, `exchange_rate`, salary, notes).
Plan	A monthly amount allocated to an Envelope inside a Month.
Funded	A Plan that has been marked as paid; creates a positive Funding movement on that date.
Skipped	A Plan that has been intentionally left for the month; creates no movement.

Term	Meaning
Direct debit	A Plan marked for direct-debit payment, shown as `DD` on the Envelope row.
Expense	An authoritative decrease of an Envelope (and optional payment Account).
Refund	An authoritative increase of an Envelope (and receiving Account) reversing an Expense.
Currency	The authoritative accounting currency of the database (one immutable value for the life of the database).
Helper currency	A configurable presentation/input currency; never a parallel authoritative currency.
Import Profile	A persisted JSON document that names and locates one recognized source statement file.
Reconciliation	Resolving imported statement rows into Expenses, Refunds, Transfers, or an explicit `IGNORE`.

Source: `docs/architecture/CURRENT_STATE.md` (Existing Business Concepts) and `docs/architecture/PRINCIPLES.md` (principles 0, 0a, 23, 25, 26).

---

1. Initial / bootstrap setup

There are three supported bootstrap paths.

1.1 Local development checkout

The minimum local run is described in `README.md` and `docs/architecture/CURRENT_STATE.md` ("Routes to instantiate FIN").

```
composer install
mkdir -p "$HOME/.config/fin"
cp config.local.example.php config.local.php
# Edit config.local.php; set absolute paths under $HOME for master_pin_file and
# throttle_file, set webauthn_rp_id='localhost' and
webauthn_origin='http://localhost:8080',
```

```
# leave file_group empty.
php api.php master-pin-status          # confirm path resolves; PIN starts as 'no'
php api.php set-master-pin             # enter + confirm a new PIN (interactive shell)
php api.php migrate                    # creates schema and DB version row
php -S 127.0.0.1:8080                  # serve the app on localhost
```

Open `http://localhost:8080`. The first browser reaching the app uses the Master PIN once and is automatically enrolled for device login (see section 4).

Source: `README.md` "Local run"; `scripts/update.sh`; `auth.php:master_pin_file` / `auth.php:set_master_pin`; `api.php:run_cli`.

1.2 Production deployment

Production is shipped as a simple PHP/SQLite installable web app. The tracked `config.php` resolves:

- `auth.master_pin_file` = `/etc/fin/master-pin.hash` (mode `0640`, group `www-data`).
- `auth.throttle_file` = `/var/lib/fin/master-pin-throttle.json` (mode `0660`).
- `webauthn_rp_id` and `webauthn_origin` as neutral placeholders (`example.com`). Each installation must override both in its ignored `config.local.php` with its own host and public origin, because a WebAuthn relying-party id only works on the host it names. Create that override **before** the first deployment of a new release, otherwise Device login stops matching the site's real origin.

The Master PIN and throttle files are server-only and live outside the web root and outside SQLite. `php api.php master-pin-status` prints only the resolved file path and `Master PIN configured: yes|no`; it never prints the hash.

`scripts/update.sh` will refuse to run unless `php api.php master-pin-status --require` exits `0`.

Source: `config.php`; `auth.php:master_pin_file` / `auth.php:fin_auth_config`; `README.md` "Production update"; `docs/architecture/CURRENT_STATE.md` "External authentication configuration".

1.3 Android install (Progressive Web App)

```
1. Open the public origin (e.g. https://your-host) in Chrome.
2. Complete Master PIN sign-in (or automatic Device login if already enrolled).
3. Open Chrome's menu → Install app (or Add to Home screen).
4. Confirm Install.
```

Production must be served over HTTPS for Android to offer the full PWA install path. `localhost` is allowed for development. Plain HTTP LAN addresses do not work for PWA install.

Source: `README.md` "Install on Android"; `manifest.webmanifest`.

---

2. Deployment and update procedure

2.1 Standard application update

```
./scripts/update.sh
```

The script performs, in order:

1. Verifies the FIN working tree has no tracked local changes (refuses otherwise).
2. Runs `php api.php master-pin-status --require` (refuses if the Master PIN is not configured; this is the production bootstrap guard).
3. If `data/fin.sqlite` exists, creates a VACUUM-INTO backup named `fin-upgrade-YYYYMMDD-HHMMSS[-N].sqlite` under `$HOME/fin-backups` (or `FIN_BACKUP_DIR`) with directory mode `0700` and file mode `0600`.
4. `git pull --ff-only`.
5. `composer install --no-dev --prefer-dist --no-interaction --optimize-autoloader` to install locked runtime dependencies.
6. `php api.php migrate` to apply only migrations that are not yet recorded.
7. Prints `FIN Planner v<AppVersion> / database v<DbVersion>`.

Optional environment overrides:

```
FIN_BACKUP_DIR=/secure/backups PHP_BIN=/usr/bin/php ./scripts/update.sh
```

Source: `scripts/update.sh`; `api.php:default_backup_dir` / `api.php:secure_backup_directory`.

2.2 Version verification

```
php api.php version                         # both app and DB versions
php api.php ?action=version               # JSON form, same data
```

The application version comes from the `VERSION` file in the web root. The database version is the highest version recorded in `schema_migrations`. Application and database versions are

independent; an application bump without a migration leaves the DB version unchanged.

Source: `api.php:app_version` / `api.php:version_info` / `api.php:database_version`; `README.md` "Versioning".

---

3. Master PIN setup and recovery

3.1 Set or change the Master PIN

The Master PIN is configured by the server-side CLI only.

```
sudo php api.php set-master-pin
```

The CLI prompts twice with terminal echo disabled. The PIN must contain 8–32 digits and is hashed with Argon2id (or the configured `PASSWORD_DEFAULT` fallback). The hash is written atomically to `auth.master_pin_file` with mode `0600` (single-user) or `0640` (group-shared). The CLI never returns the PIN or its hash.

For protected input streams the CLI accepts `--stdin`, reading the PIN and confirmation from two lines. Interactive shell echo is required when `--stdin` is not used.

Source: `api.php:run_cli` `set-master-pin`; `auth.php:cli_read_master_pin`; `auth.php:master_pin_password_hash`; `auth.php:set_master_pin`.

3.2 Status check

```
php api.php master-pin-status             # prints resolved path + yes/no
php api.php master-pin-status --require   # exit code 1 when PIN is not configured
```

`update.sh` uses `--require` to fail fast before touching the database.

Source: `api.php:run_cli` `master-pin-status`; `scripts/update.sh`.

3.3 Throttle behaviour

A persistent progressive throttle lives in `auth.throttle_file`:

- Each unsuccessful PIN attempt increments a counter.
- After five consecutive failures, the CLI delays the response; the delay doubles per additional failure, capped at 900 seconds.
- A correct PIN resets the counter to zero.

- `set-master-pin` resets the counter as a side-effect of successful configuration.

A successful device login clears the throttle completely, and a successful Master PIN login resets the counter once the block has expired. To clear the throttle without changing the PIN, delete the configured `throttle_file` (shell access required).

Source: `auth.php:auth_throttle_delay` / `auth.php:verify_master_pin` / `auth.php:reset_master_pin_throttle` / `auth.php:webauthn_verify_assertion`; `api.php` action `masterPinLogin`.

3.4 Recovery (lost PIN, lost browser, lost throttle)

The Master PIN is not stored in SQLite and is not exposed through any web API. Recovery requires shell access on the server.

Symptom	Recovery step
Forgot PIN	`sudo php api.php set-master-pin` to overwrite the hash with a new PIN.
Lost access to all enrolled browsers	`sudo php api.php set-master-pin` keeps the hash; sign in once with the PIN, the next browser becomes the new Device login.
Locked out by a long throttle and a forgotten PIN	Delete the configured `auth.throttle_file` and then run `set-master-pin`.

Source: `auth.php:master_pin_hash`; `api.php:run_cli`.

---

4. WebAuthn / Device login enrollment and recovery

4.1 Automatic enrollment

The first time a browser reaches FIN with no FIN session and no `fin.deviceCredentialId` in local storage, the sign-in screen shows **Master PIN**.

1. The user enters the Master PIN.
2. The web app calls `api.php?action=webauthnRegistrationOptions`.
3. The web app calls `navigator.credentials.create({ publicKey })` for a non-discoverable platform credential (Windows Hello, Android device verification, iOS keychain, or equivalent).

4. The verification succeeds; the credential ID is stored locally as `fin.deviceCredentialId` and the same FIN session is opened.

There is no enrollment button and no passkey discoverable flow.

Source: `app.js:loginWithMasterPin` / `app.js:enrollCurrentDevice`; `api.php` actions `webauthnRegistrationOptions` and `webauthnRegistrationVerify`; `auth.php:webauthn_*`.

4.2 Daily authentication

On every subsequent visit:

1. The web app sends the locally remembered `fin.deviceCredentialId` with the request to `api.php?action=webauthnAssertionOptions`.
2. If the credential still exists and is enabled, the platform prompts for verification (face/fingerprint/PIN).
3. The server verifies the assertion with required user verification, short-lived challenges, and the configured RP ID / origin.

If the local credential is missing, revoked, or revoked server-side, it is removed from local storage and the user is taken to the Master PIN form.

Source: `app.js:beginAuthentication` / `app.js:showAuthState`; `docs/architecture/CURRENT_STATE.md` "Authentication View" / "Existing Modules".

4.3 Cancellation and failure states

State	User-visible message	Buttons
Cancelled by user (NotAllowedError)	"Authentication cancelled"	Try again / Use Master PIN
Generic failure	"Device authentication failed"	Try again / Use Master PIN
PIN not configured and no device login	"Authentication must be configured by the FIN administrator."	(server-only CLI required)

Source: `index.html` (auth view) and `app.js:showAuthState`.

4.4 Listing, renaming, and revoking credentials

After signing in, open the floating header **Security shield** button (shield icon between the logout × and the version label, visible only after sign-in).

The **Device login** dialog opens with one row per active credential. Each row shows the credential Label, "Enabled", the date of last successful use, and two buttons:

- **Rename** — change the Label.
- **Remove** — revoke the credential; the next browser session for this credential will fall back to Master PIN.

Removing a credential revokes the server-side record but does not affect FIN data.

Source: `index.html` `securityDialog`; `app.js:openSecurityDialog` / `app.js:renameDeviceCredential` / `app.js:revokeDeviceCredential`; `api.php` actions `renameWebauthnCredential` and `revokeWebauthnCredential`.

4.5 Logout

The left × button in the floating header logs the current browser out and clears the local FIN session. It does **not** revoke the device credential; the same browser still holds `fin.deviceCredentialId`. Use the Security shield to remove the credential.

Source: `app.js:logout`; `index.html` `logoutBtn`.

4.6 Host or origin change

WebAuthn credentials are bound to the configured `webauthn_rp_id` and `webauthn_origin`. After moving FIN to a new host or changing the configured origin, all device credentials remain in SQLite but are no longer usable by browsers. Set the new values in the ignored `config.local.php` (the tracked defaults are only placeholders). Recovery:

1. Sign in once with the Master PIN on a new browser.
2. The new browser is automatically enrolled.
3. Optionally use the Security shield to rename the new credential and remove the old ones.

Source: `auth.php:webauthn_server` / `auth.php:expected_webauthn_origin`; `README.md` "Production update" (host-change note).

---

5. Backup and restore

5.1 What FIN writes into backups

- `scripts/update.sh` writes a VACUUM-INTO copy of `data/fin.sqlite` to `fin-upgrade-YYYYMMDD-HHMMSS[-N].sqlite` (mode `0600`).
- `php api.php archive --keep-months=12` writes a complete SQLite backup plus a SHA-256 JSON manifest named `fin-archive-before-YYYY-MM-DD-TIMESTAMP-*.sqlite` (mode `0600`).
- `php api.php refresh-snapshot [--backup-dir=PATH]` writes a read-only mode-`0600` snapshot named `fin-prod-dev-refresh-YYYYMMDD-HHMMSS-RANDOM.sqlite`. This is used by `./scripts/refresh-dev-from-prod.sh` for the one-way PROD → DEV operator workflow.
- `scripts/refresh-dev-from-prod.sh` (when run on the development checkout) writes the pre-refresh development database to `fin-dev-before-prod-refresh-*.sqlite` (mode `0600`).

All four share `$HOME/fin-backups` (or the path in `FIN_BACKUP_DIR`) with directory mode `0700`. Override per call with `--backup-dir=PATH` for `archive` and `refresh-snapshot`.

Source: `api.php:default_backup_dir` / `api.php:secure_backup_directory` / `api.php:create_refresh_snapshot` / `api.php:archive_and_cleanup_database`; `scripts/update.sh`; `scripts/refresh-dev-from-prod.sh`.

5.2 What the backups do not contain

- The Master PIN hash file is **not** included in the SQLite backup.
- The Master PIN throttle file is **not** included in the SQLite backup.
- Local configuration under `config.local.php` is gitignored and not part of the snapshot.
- Statement files under `import/` are not part of the snapshot. Import Profiles that have been saved through `save-import-profile` are stored in SQLite and travel with the database backup.

Source: `docs/architecture/CURRENT_STATE.md` "External authentication configuration"; `api.php:save-import-profile`.

5.3 Restore a backup over the live database

`scripts/restore-backup.sh` restores any FIN SQLite snapshot — `fin-upgrade-*.sqlite`, `fin-archive-before-*.sqlite`, `fin-prod-dev-refresh-*.sqlite`, or `fin-dev-before-prod-refresh-*.sqlite` — over the live database.

Use it when the live database must be brought back to an earlier known-good point: after a bad update, a bad import or correction, a mistaken history cleanup, or a corrupted live database file.

Prerequisites

1. **Stop the web server and every other process that opens the database.** The script prints this notice before it does anything:

`text NOTICE: stop the web server (and any other process that opens the SQLite database) before running this script. The script will refuse to run while another process has the live database or its -wal/-shm/-journal sidecars open; it cannot detect every interpreter on every host.`

The script additionally refuses while the live database or one of its sidecars is open (see "Refusals" below), but that check only catches a long-lived writer. A request worker holds the file only for the duration of one request, so the check usually finds nothing even while the application is in use. Stopping the application is the safe default. 2. **Run the script as the account that owns the live database file.** Ownership and permissions of the restored file are taken from the live file, so the operator must be able to apply them. When the live database is owned by a different account, run the script as that account (or with sufficient privilege) instead. 3. **Have the backup file readable** by that account. The backup is copied, never moved.

Usage

```
./scripts/restore-backup.sh <backup-file>
```

Optional environment overrides:

```
FIN_DB_FILE=/path/to/live/fin.sqlite FIN_BACKUP_DIR=/path/to/backups \
  ./scripts/restore-backup.sh <backup-file>
```

`FIN_DB_FILE` defaults to `data/fin.sqlite`; `FIN_BACKUP_DIR` defaults to `$HOME/fin-backups` and is where the pre-restore safety copy is written.

What the script does

1. **Validates a copy, never the original.** The backup is copied to a stage file in the live database's directory. `PRAGMA integrity_check` and a schema-version comparison run on that copy. The original backup file is never consumed and stays byte-identical.
2. **Refuses** a missing file, a file that is not a SQLite database, a file that fails `PRAGMA integrity_check`, a backup whose schema version is newer than the running code, and a live database that is open by another process. Every refusal prints one `Error:` line and exits with status 1; nothing is replaced.
3. **Writes a pre-restore safety copy** of the current database under `$HOME/fin-backups` (or `FIN_BACKUP_DIR`), mode `0600`, named `fin-restore-before-YYYYMMDD-HHMMSS[-N].sqlite`. The copy is a `VACUUM INTO` snapshot when the live database is readable; when it is not, the script falls back to raw copies of the database file and any `-wal` / `-shm` / `-journal` sidecars named `fin-restore-before-YYYYMMDD-HHMMSS-raw.{sqlite,-wal,-shm,-journal}`. When no live database exists yet, the script reports that and skips the safety copy.
4. **Removes stale `-wal`, `-shm` and `-journal` sidecar files** for the target database, so the replaced file does not inherit journal state from the previous database.
5. **Replaces the database** by renaming the staged copy over the live file inside the same directory, which is atomic on one filesystem.
6. **Preserves the live file's owner, group and mode.** The attributes are captured before anything is replaced and applied to the restored file; `chown` and `chgrp` are skipped when the staged file already has the target owner and group. When no live database existed, the script falls back to mode `0660` (group-writable so the web server can write), the group of the live database's directory, and the current user as owner.
7. **Prints the result and the post-restore checks** (below).

The restore prints, in this order:

```
Restored: <live-database> (owner=<owner> group=<group> mode=<mode>)
Original backup file kept in place: <backup-file> (not consumed)
```

Post-restore checks

The script runs two checks for you and prints their output:

```
Post-restore version:
FIN Planner v<AppVersion> / database v<DbVersion>

Post-restore Master PIN status:
Master PIN file: <path>
Master PIN configured: yes|no
```

The version line confirms which application and database version the restored file is on. An older backup brought forward is migrated by the running code on the next open.

The Master PIN status is informational: the PIN hash is not part of the SQLite backup (§5.2), so a restored database keeps the PIN state of the installation, not of the backup.

After the restore, start the web server again and confirm the application opens and shows the expected Months, Envelopes, Accounts, and Transactions.

Reverse a restore with the safety copy

The restore is itself reversible. When a live database existed, the script keeps both the safety copy and the original backup and prints:

```
Both safety copy and original backup are kept:
  safety copy: <safety-copy> (kind: vacuum)
  backup file: <backup-file>
If this restore is unwanted, reverse it with:
  scripts/restore-backup.sh <safety-copy>
```

To undo the restore, stop the web server (prerequisite 1) and run the same command with the safety copy in place of the original backup. The safety copy is an ordinary backup file and is restored by the same procedure, with its own safety copy written first.

What a restore does not bring back

A restore replaces the SQLite database only. It does not bring back:

- The Master PIN hash file (`config.php['auth']['master_pin_file']`).
- The Master PIN throttle file (`config.php['auth']['throttle_file']`).
- Local configuration under `config.local.php` (gitignored).
- Raw statement files under `import/` (already-imported files remain in `import/imported/`). Saved Import Profiles in `statement_import_profiles` are stored in

the database and travel with the backup.

- Any files outside the database directory (e.g. application code, `vendor/`, `node_modules/`).

See also section 5.2, "What the backups do not contain".

Refusals

Every refusal prints a single `Error:` line naming the cause and exits with status 1. The live database is left untouched.

Cause	`Error:` message	Exit status
Backup file missing	`Error: Backup file not found: <backup-file>`	1
Backup file is not a SQLite database	`Error: Backup file is not a SQLite database: <backup-file>`	1
`PRAGMA integrity_check` failure	`Error: Backup file validation failed: integrity-check:SQLSTATE[HY000]: General error: 11 database disk image is malformed`	1
Backup schema newer than the running code	`Error: Backup schema version (<backup-version>) is newer than the running code (<code-version>).`	1
Live database open by another process	`Error: live database <live-database> is open by process(es) pid=<pids> command=<commands>; stop the web server and any other process that uses <live-database> before retrying`	1

The integrity-check line carries the SQLite message, so the wording after `Backup file validation failed:` depends on the damage that was found. The script also refuses a backup path that is not a regular file (`Error: Backup path is not a regular file: <backup-file>`) and one that is not readable (`Error: Backup file is not readable: <backup-file>`).

Source: `scripts/restore-backup.sh`; `docs/architecture/CURRENT_STATE.md` "Restore Workflow"; `tests/shell/restore-backup.test.sh`.

---

6. Account operation

6.1 Add an Account

1. Open **Month → Accounts**.
2. Tap **Add account** (or any existing Account row).
3. Fill the dialog:

Field	Notes
Name	Display only. Not used as identity.
Account number	Stable identifier; required. Used for statement Import matching and exact counterpart-Account recognition.
Currency presentation	`CURRENCY` or `HELPER_CURRENCY` — informational only; never reinterprets finance.
Initial Balance	Optional. Blank leaves the Balance as **Unknown**. Any signed amount, including zero, creates a same-day `account_creation` observation.

1. Save. The new Account appears in the list with its current signed Balance (or "Unknown").

Source: `app.js:openAccountDialog`; `accounting.php:create_account_with_initial_position`; `api.php` action `saveAccount`; `docs/architecture/CURRENT_STATE.md` "Accounts".

6.2 Edit an Account

Open the Account in the dialog and modify Name, Account number, or Currency presentation. Renaming never alters financial history; the Account number is the stable identity. The Currency presentation field changes how amounts display for this Account and does not rewrite stored amounts.

Source: `accounting.php:update_managed_account`; `api.php` action `saveAccount`.

6.3 Account Balance — Known and Unknown

The Balance for an Account on date D is:

- The latest signed absolute Balance observation on or before D (covering same-day movements), plus ordinary movements strictly after that observation through D.
- Without any observation, Balance is explicitly `null` / "Unknown" — even when movements exist.

Transactions, Refunds, Income, Transfers, and Reconciliation remain valid at any date and never require Balance knowledge.

A movement dated on the observation date is already inside the observed end-of-day balance and is never added again; record a new observation on a later date when that day's spending was booked after the observation was taken.

Source: `accounting.php:account_balance_state_as_of` / `accounting.php:account_balance_as_of`; `docs/architecture/CURRENT_STATE.md` "Accounts".

6.4 Record Balance observation

Inside an open Account dialog:

1. Pick a non-future effective date.
2. Enter a signed Balance amount (negative, positive, or zero).
3. Optional note.
4. Save.

One observation per Account/date is upserted in place. Recording an observation never creates an opening, reconciliation, or correction movement.

Source: `api.php` action `saveAccountBalanceObservation`; `accounting.php:record_account_balance_observation`; `app.js:reconcileCurrentAccount`.

6.5 Delete Balance observation

The history view lists observations alongside ordinary movements. Deleting an observation restores the next-latest observation as the effective Balance (or Unknown when no other observation applies). It never inserts a reversal movement.

Source: `api.php` action `deleteAccountBalanceObservation`; `accounting.php:delete_account_balance_observation`; `app.js:deleteAccountBalanceObservation`.

6.6 Archive and Restore

Open an Account and tap **Archive account**. Archived Accounts:

- Remain in history and may be selected for display.
- Disappear from the new-expense "Paid from" default chooser.
- Keep their existing default-Account association in Envelope management; an archived Account cannot be chosen as a new default.

Tap **Restore account** to bring it back. The History view and Balance history are preserved across archive/restore.

Source: `accounting.php:set_account_active`; `api.php` action `setAccountActive`; `app.js:toggleCurrentAccountActive`.

6.7 Account-to-Account Transfers — *Gap*

No general manual Account-to-Account Transfer action is exposed in the UI today. Use statement reconciliation (section 8) to record Account-to-Account Transfers.

Source: `index.html` (no Transfer dialog control); `app.js` (no Transfer handler); `docs/architecture/CURRENT_STATE.md` (open gap).

---

7. Envelope operation

7.1 Add or edit an Envelope

1. Open **Month → Envelopes**.
2. Tap **Add envelope** or an existing Envelope name.
3. Fill the dialog:

Field	Notes
Name	Required. Unique after trimming and ignoring capitalization.
Default Account	Optional. Convenience for new-expense "Paid from" prefill only. Never inferred from a legacy label.
Role	`Daily spend` (always visible on Today), `Reserved`, or `Savings`.
Preferred currency	`CURRENCY` or `HELPER_CURRENCY` for the editing input.

Field	Notes
Visibility (Reserved/Savings only)	`Normal`, `Hide balance`, or `Show on Today`. Daily spend is fixed `Normal`.
Savings goal (Savings only)	Optional target amount.
Goal-progress visibility (Savings only)	`Hidden`, `Month only`, or `Today and Month`.
Manual historical baseline	Optional user-confirmed external reference in Currency (advisory only, never synthesised).
Fixed	Optional. Keeps the monthly amount you set; FIN never recommends a different one for it.
Monthly reset	Optional. Starts the new month with exactly that month's allocation instead of carrying the previous balance over.
Note	Optional.

Fixed and **Monthly reset** are independent. A Fixed Envelope keeps the amount you set and never receives a FIN funding recommendation; the amount stays editable by hand. An Envelope with **Monthly reset** starts each new month with exactly that month's allocation, so a leftover from the previous month is not carried over. The reset happens when that month's allocation is funded; an amount you move forward for a later month is only added to the Envelope. Both checkboxes are off by default, so an Envelope behaves exactly as before until you change it.

1. Save. The Envelope appears in the Envelopes list with its current Balance.

Source: `app.js:openEnvelopeDialog`; `api.php` action `saveEnvelope`; `api.php` actions `saveEnvelope` / `deleteEnvelope`.

7.2 Delete an Envelope

Deleting an Envelope removes it from the global list and its monthly Plans for future Months. Past history is preserved through transactions and movements.

Source: `api.php` action `deleteEnvelope`; `api.php:delete_envelope`.

7.3 Envelope Balance — Known and Unknown

The Envelope Balance mirrors the Account Balance model.

- The latest signed absolute Balance observation on or before the cutoff, plus movements strictly after that observation through the cutoff.
- Without any observation, the API falls back to the ordinary movement sum from zero.
- Negative values are valid Balances and remain visible on Today.

Source: `docs/architecture/ENVELOPE_MODEL.md`; `docs/architecture/CURRENT_STATE.md` "Envelope Balance observation model".

7.4 Record Balance observation

Open the Envelope dialog, then the Balance tools button.

1. Pick a non-future effective date.
2. Enter a signed Balance amount.
3. Optional note.
4. Save.

Recording never creates a movement, transfer, transaction, or correction event. One observation per Envelope/date is upserted in place.

Source: `api.php` action `saveEnvelopeBalanceObservation`; `api.php:record_envelope_balance_observation`; `app.js:recordCurrentEnvelopeBalance`.

7.5 Delete Balance observation

The Balance tools history lists observations and ordinary movements together. Deleting an observation restores the next-latest observation as the effective Balance (or the movement-sum-from-zero fallback). It never inserts a reversal movement and never undoes a Transfer, Expense, Refund, or Funding record.

Source: `api.php` action `deleteEnvelopeBalanceObservation`; `api.php:delete_envelope_balance_observation`.

7.6 Envelope-to-Envelope Transfers

Open an Envelope dialog → Balance tools.

1. Choose **Transfer** to another Envelope.

2. Enter amount, date, optional note.
3. Confirm two confirmations when applicable:
4. If the transfer would make the source Envelope negative, confirm explicitly.
5. If the two Envelopes point to different default Accounts, confirm that the real bank transfer has been completed.
6. Save. The transfer creates linked `transfer_out` / `transfer_in` movements whose sum is zero.

Transfers can be undone from the Balance tools history view (paired reversal).

Source: `app.js:transferFromCurrentEnvelope`; `api.php` action `transferEnvelope` / `undoEnvelopeTransfer`; `api.php:transfer_envelope_balance` / `api.php:undo_envelope_transfer`.

---

8. Statement import and reconciliation

8.1 Add an Account first

Open **Month → Accounts** and create the Account that owns the statement (see section 6.1), then select that Account for the import. The selected Account is authoritative: an Account identifier found in the statement file is informational evidence only and never selects, validates, or rejects the Account.

Source: `docs/architecture/CURRENT_STATE.md` "Accounts"; `docs/architecture/STATEMENT_IMPORT_RECONCILIATION_ADDENDUM.md`; `statement_import.php:statement_account` / `statement_import.php:preview_statement_import`.

8.2 Drop a statement file (CLI path)

FIN ships an `import-statements ACCOUNT_ID` CLI that reads every regular file in `<FIN root>/import/`, parses each through the bundled structural signatures and any saved Import Profiles, writes successful rows to `statement_imports` / `statement_rows`, and moves the source file to `<FIN root>/import/imported/`. Files must be valid XLSX, ≤ 5 MB, and ≤ 10 000 rows.

The command exits `0` on success and `1` when any file failed.

The CLI itself is implemented and exercised end-to-end by the integration test suite (`tests/php/StatementFileDropTest.php` and `tests/php/StatementImportRealFilesTest.php`). It is **not** currently presented as a single end-to-end operator procedure in this manual because the operator must first have at least one saved Import Profile (§8.5); without one, every drop

fails with `No matching Import Profile`.

CLI references:

```
php api.php inspect-statement FILE                 # print Import Profile header candidates as JSON
php api.php import-statements ACCOUNT_ID           # process all dropped files
```

Source: `api.php:run_cli` `import-statements` / `inspect-statement`; `statement_import.php:statement_drop_dir` / `statement_import.php:statement_process_dropped_files`; `tests/php/StatementFileDropTest.php`; `tests/php/StatementImportRealFilesTest.php`.

8.3 Import through the web UI

Open **Month → Accounts → Import statement**.

1. Select the active Account to claim the import.
2. Select the XLSX file (≤ 5 MB).
3. Tap **Preview**. FIN returns the inert-evidence preview (no movements are written).
4. Tap **Import evidence**. The raw rows are persisted in `statement_imports` / `statement_rows`. The upload is financially inert until rows are resolved through reconciliation.

For each import, available Import Profiles (`save-import-profile`, `list-import-profiles`, `enable-import-profile`, `disable-import-profile`, `rename-import-profile`, `delete-import-profile`) can be managed through the CLI; the web UI will match an upload to a saved profile by structural signature.

Source: `app.js:openStatementImportDialog` / `app.js:previewStatementImport` / `app.js:importStatementEvidence`; `api.php` actions `previewStatementImport` / `importStatementEvidence`; `docs/architecture/STATEMENT_IMPORT_RECONCILIATION_ADDENDUM.md`.

8.4 Reconciling rows

After importing rows, open **Month → Accounts → Reconcile** (or the matching control in the Accounts section). FIN loads the unresolved rows for the chosen Account newest first, by booking date descending; up to 32 are loaded at a time, and the queue refills after each Save.

- Rows that already match an authoritative Expense, Refund, or Account event (and whose match is unambiguous) are resolved automatically right after the import,

before the manual queue opens.

- A planned **Direct debit** payment and the real payment that arrives later as statement evidence are one and the same payment. When the allocation has no "Paid from" Account, the automatic entry is recognised anyway and takes the Account the statement row came from. When the planned and the real amount differ, the automatic entry takes the real amount. Several real debits that claim one planned payment stay in the manual queue instead of being split.
- Rows that cannot be resolved deterministically need a manual classification per row:
- Choose an **Envelope** for an Expense or Refund.
- Choose an **Account** for a Transfer.
- Choose **IGNORE** for a durable "no FIN action" disposition.

Save submits only the rows with a chosen classification. Errors are shown inline; re-open the dialog to retry any remaining rows.

A row whose Expense or Refund would repeat an existing entry exactly — the same date, Envelope, amount and description — is not written on the first Save. FIN names the existing entry and asks once; confirming writes the row anyway. The question is never automatic, because identical same-day payments are sometimes legitimate.

Source: `app.js:openReconciliationDialog` / `app.js:renderReconciliationRows`; `api.php` action `batchReconcileStatementExpenses`; `statement_reconciliation.php`.

8.5 Import Profile management (CLI)

```
php api.php validate-import-profile PROFILE.json
php api.php preview-import-profile PROFILE.json FILE
php api.php save-import-profile PROFILE.json
php api.php list-import-profiles
php api.php rename-import-profile ID NAME
php api.php enable-import-profile ID
php api.php disable-import-profile ID
php api.php delete-import-profile ID
```

`PROFILE.json` must be valid UTF-8 JSON no larger than the schema's max bytes; the CLI refuses values that exceed the limit.

Source: `api.php:run_cli`; `docs/architecture/FIN_IMPORT_PROFILE_ARCHITECTURE_DESIGN_2026-09-07.md`.

---

9. Other day-to-day procedures

9.1 Add expense

Open the **Add expense** button (always-visible inside the safe-to-spend card on Today) or from Recent expenses on Today.

Field	Behaviour
Amount	One field; follows the Envelope's `CURRENCY` or `HELPER_CURRENCY` preference. Decimal separator is a comma.
Envelope	Preselected as the most-used visible Envelope on Today; out-of-sight Envelopes are listed in a separate group.
Note	Optional.
Date	Optional; defaults to today.
Paid from	Default Account; overridable. Archived Accounts are not offered.

Save persists one authoritative Currency amount (Helper input is converted through the selected Month's `exchange_rate`). The expense appears in Recent expenses on Today and in the Envelope movements.

If an identical expense already exists — the same date, Envelope, amount and description — FIN says so and asks before writing it again. The answer is never automatic: two identical payments on the same day are sometimes legitimate.

Source: `app.js:saveExpense`; `api.php` expense actions; `docs/architecture/CURRENT_STATE.md` "Today Screen".

9.2 Edit or delete a manual / effective expense

Tap a row in Recent expenses. The dialog opens with **Edit expense** / **Edit Refund** title. Delete is offered inside the same dialog.

Future scheduled Direct debit expenses (DD rows with `spent_date` in the future) are read-only; tapping them links to the source allocation instead of opening the edit dialog. Once the

`spent_date` arrives, the same row becomes editable/deletable like any other Actual Expense.

Source: `app.js:openEntryDialog`; `app.js:editExpenseAllocationBtn` / `app.js:deleteCurrentExpense`; `docs/architecture/CURRENT_STATE.md` "Today Screen".

9.3 Add Income (salary)

Open **Month → Month** section, set the monthly salary in the Month form, save. Income is Month-level metadata, not a transaction.

Source: `app.js:saveMonth`; `docs/architecture/CURRENT_STATE.md` "Month".

9.4 Add Month

Open **Month → Month** section, use the **Add Month** control. The new Month opens the form with empty year/month selectors; save it after entering the year, month, salary, exchange rate (Helper × rate = Currency).

Source: `app.js:saveMonth` / `app.js:openMonthForm`; `api.php` action `saveMonth`.

9.5 Copy Month

Open **Month → Month → Copy Month**. Pick a source Month and a target year/month. Copy keeps Skipped rows Skipped, resets every other Plan to Planned, clears funding dates, preserves target Expenses, and uses the destination Month's own `exchange_rate`.

Source: `app.js:copyMonth`; `api.php` action `copyMonth`.

9.6 Plan editing and the P/F/S cycle

Inside **Month → Funding**, tap the Plan amount to edit the selected Month's Plan. Tap the right-most cell to cycle Planned → Funded → Skipped → Planned.

- **Planned**: no movement is written.
- **Funded**: a positive Funding movement is added on the action date (today, unless the row has an explicit due date).
- **Skipped**: no movement is written.

Source: `api.php:set_allocation_status` / `api.php` action `setAllocationStatus`; `app.js:cycleAllocationStatus`.

9.7 Direct debit allocations

When a Plan is marked as Direct debit (DD), a Paid-from Account may also be chosen per row. Marking a DD row **Funded** does three things on the action date:

1. Adds a positive Funding movement.
2. Creates a scheduled authoritative Transaction dated on the DD row's payment date (defaults to the first day of the allocation's own Month if empty).
3. The Transaction becomes effective on its `spent_date` and is treated as an ordinary Actual Expense from then on. Until that date, the row is read-only.

Changing away from Funded removes the future-derived records; effective Expenses persist independently.

Source: `docs/architecture/ENVELOPE_MODEL.md` (Month Plan Relationship section); `docs/architecture/CURRENT_STATE.md` "Month Screen".

9.8 Reports (PDF / XLSX)

Open **Month → Month → Report / export**. Pick the from-month and to-month (inclusive), then download the **PDF** or **XLSX**:

- The **PDF** separates plans, funding, actual transactions, role-specific envelope activity, cutoff-effective closing balances, and user-meaningful transfers. It is Unicode-printable.
- Each month shows its own Planned income, Plan, Plan difference, and Actual spending, and every role table (Daily Spend, Reserved, Savings) shows Plan, Funded, and Actual next to the per-Envelope detail.
- A month that has at least one Income event also shows **Recorded income** — the sum of that month's Income events — and an **Income difference** (Recorded income minus Planned income). A month without an Income event shows neither figure, so income that was simply never reconciled never reads as zero. Planned income and Recorded income stay separate numbers; neither is derived from the other.
- A range of two or more months adds a **range summary** above the Daily Spend chart: the range totals for Plan, Funded, Actual spending, and Difference, plus the same Plan / Funded / Actual split per role. A single-month export has no range summary.
- The **XLSX** retains the detailed single-worksheet, months-stacked layout. Each month block also carries the same **Actual spending** total the PDF prints and a **Plan vs Actual difference** (Plan minus Actual spending) next to the Income / Plan / Plan difference rows. A month block whose month has at least one Income event also carries a **Recorded income** row and an **Income difference**, with the same

values the PDF prints.

Both use the configured `currency` for authoritative amounts and the selected Month's `exchange_rate` for Helper conversions; a missing Helper rate for a transaction date causes the conversion to fail rather than fall back to a cross-Month rate.

Source: `app.js:downloadReport`; `report.php:build_report_pdf` / `report.php:report_income_events_by_month`; `api.php:build_report_xlsx`; `docs/architecture/CURRENT_STATE.md` "Existing Business Concepts" → "Report".

9.9 Funding Review

Open **Month → Month → Report / export → Funding Review** to compare the current Plan with deterministic recommendations learned from completed Transaction history. Manual historical baselines are separate advisory references and never become synthetic history. A Fixed Envelope is skipped: its monthly amount is your own, so FIN derives no amount for it and suggests none.

A recommendation follows the shape of the spending. A stable monthly obligation keeps its observed amount, a regular but variable month is estimated from its median and a recency-weighted average, and spending that is not a monthly recurrence — a quarterly or annual bill, an irregular household cost — is funded by its average monthly cost. Such an obligation is never recommended as a monthly requirement, and an Envelope with real but irregular spending is never recommended as 0.

In the Plan editor, the historical baseline is shown in the currency the Envelope plans in, with the Currency amount kept as the secondary value. Funding Review itself compares Current Plan, Baseline, FIN recommendation, Assessment, and Confidence in Currency, because those columns are compared with each other.

Source: `recommendation.php`; `app.js:openFundingReview`; `api.php` action `fundingRecommendation`.

9.10 API token (external client)

```
sudo php api.php create-api-token
```

The CLI generates a random token, writes only its SHA-256 hash to `auth.api_token_file` (mode `0600`, or `0640` with the configured file group), and prints the token once. Store it in the client's configuration; FIN never stores the token itself and never prints or logs the stored hash.

The client sends it as `Authorization: Bearer <token>`. The token authorizes:

- five read actions on a GET request: `action=history`, `action=transactions`, `action=transfers`, `action=fundingRecommendations`, and `action=accountBalances`;
- one write action on a POST request: `action=setPlanAmounts` (see 9.13).

Every other action and every other POST still requires the ordinary browser session, the token creates no session, and no CSRF token is involved. A wrong or missing token gets the usual 401 response, and no Master PIN throttle is used.

Running the command again replaces the stored hash and revokes the previous token. There is no status, list, or delete command.

Source: `api.php:run_cli` `create-api-token`; `auth.php:create_api_token` / `auth.php:api_token_authorized` / `auth.php:api_token_write_authorized`.

9.11 History API (external client)

```
GET api.php?action=history&from=YYYY-MM&to=YYYY-MM
```

Returns JSON for the selected month range. The response carries `from`, `to`, `currency`, `months`, and `savings_goals`.

Every month carries `year`, `month`, `label`, `period` (`past` / `current` / `future`), `cutoff`, a `coverage` marker, the Daily/Reserved/Savings rows and totals, `overview`, and the month's actual `income`.

The month's `income` carries `total` and the `events` behind it. It is the same Income account-event ledger and the same bucketing rule the PDF and XLSX report use for **Recorded income**, so the report and this API never disagree about one month.

`coverage` says what the month contains:

- `full` — a retained month. Its rows carry `planned`, `funded`, `actual`, `difference`, and `balance`, exactly as the PDF report computes them, and `overview` carries `planned_allocations` and `actual_expenses`.
- `actual_only` — a month that exists only as recorded spending, for example a month from 2025 that is no longer part of the retained running data. Its rows carry only `envelope_id`, `envelope`, and `actual`, its totals carry only `actual`, and `overview` carries only `actual_expenses`, because Plan, Funded, and Balance do not exist for such a month.

The actual spending of an `actual_only` month is the same actual the Funding Review recommendations use, so the history and Funding Review read actions never disagree about one month. A range that contains neither a retained month nor any recorded spending returns the usual 404 "No months found in selected range".

The current month additionally carries `safe_to_spend`: the figure the Today screen shows, with every input and intermediate result behind it. Past and future months never carry it.

`safe_to_spend` carries:

- `amount` — the figure itself: the number Today shows, rounded down to a whole 10 and never negative;
- `label` — `safe to spend today`, `over plan`, or an empty string;
- `as_of` — the date the balances and the remaining days were taken from;
- `daily_total` — the sum of the Daily spend envelope balances;
- `daily_envelopes` — the contributing envelopes (`envelope_id`, `envelope`, `balance`) in the order their balances were summed, so `daily_total` is the sum of exactly those values;
- `days_in_month` and `remaining_days` — the days in the month, and the days that remain in it, today included;
- `raw_per_day` — `daily_total` ÷ `remaining_days`;
- `rounded_amount` — `raw_per_day` rounded down to a whole 10, and 0 when `raw_per_day` is not positive;
- `rounding` — the rule as a formula;
- `zero_reason` — `null`, or the condition that produced a 0: `negative_daily_total` (the Daily total is below zero, so the label is `over plan`), `zero_daily_total` (the Daily total is exactly zero), or `below_rounding_step` (the per-day amount is positive but smaller than the rounding step of 10).

Only Daily spend envelopes contribute; a Reserved or Savings balance never does, no matter how it is presented on Today. The Daily total is the persistent envelope balance as of `as_of`, not the month's plan. The figure is the same number the Today screen and the PDF report show, so it can be reproduced by hand from the listed inputs alone.

`savings_goals` lists the Savings envelopes that have a goal: `envelope_id`, `envelope`, the target `goal`, the envelope's current `balance`, and the derived `progress_percent` (Balance ÷ goal, never negative, possibly above 100%). FIN stores no goal deadline, so none is reported.

Source: `api.php:build_history_response`; `api.php:history_actual_only_month`; `api.php:history_safe_to_spend`; `api.php:history_savings_goals`; `report.php:report_safe_to_spend`; `report.php:report_income_events_by_month`; `recommendation.php:funding_actual_spending_by_month`.

9.12 Transactions API (external client)

```
GET api.php?action=transactions&from=YYYY-MM&to=YYYY-MM[&envelope_id=ID]
```

Returns the individual authoritative Expense and Refund Transactions in the inclusive calendar-month range. `envelope_id` is optional and restricts the result to one exact Envelope. The response carries `from`, `to`, and `transactions`. Each transaction carries:

- `id`
- `date` — the authoritative `spent_date`
- `envelope_id`
- `envelope`
- `description` — the transaction note
- `amount` — Expense is positive; Refund is negative, matching the PDF's signed actual-spending convention
- `kind` — `expense` or `refund`
- `account` — the transaction's actual payment Account display name, or null when the Account is unknown

The range may contain at most three calendar months. A longer range returns HTTP 400 with `Transaction range cannot exceed 3 months`.

The endpoint uses the PDF report's selection rules: the calendar month of `spent_date` is authoritative, future-dated Transactions are excluded, Refunds are included, and Transfers, Envelope Transfers, Funding, and adjustments are not part of the Transaction list.

Source: `api.php:build_transactions_response`; `api.php:report_transaction_rows`.

9.13 Plan amounts API (external client)

```
POST api.php
Authorization: Bearer <token>
Content-Type: application/json
```

```
{"action":"setPlanAmounts","year":2026,"month":11,
 "amounts":[{"envelope_id":1,"amount":4000}]}
```

Sets the planned amount of Plan rows for one Month. It is the only write the API token may perform (see 9.10); the ordinary FIN session may call it as well.

The request carries `year`, `month`, and a list of `amounts`. Every amount row carries `envelope_id` and `amount`. The amount is an absolute amount in the installation's Currency (DKK); there is no currency field. It must be zero or more with at most two decimals. One request carries at least one and at most 500 rows, and an Envelope may appear only once.

Rules:

- The Month must already exist. The action never creates a Month; an unknown Month returns HTTP 404 `Month not found`.
- Only the listed Envelopes change. Every other Envelope of that Month keeps its Plan.
- An existing Plan row must be Planned. A Funded or Skipped row is rejected with HTTP 409 and the error names the row.
- A Plan row that does not exist yet is created as a plain Planned row: no Direct debit, no payment date.
- The action never changes a row's status or any other field, and never creates a Funding movement, a Transaction, or a Balance change.
- The request is all-or-nothing. If one row is invalid, nothing is written and the error names that row.

The response carries `ok`, `year`, `month`, `currency`, and `rows`. Every row carries `envelope_id`, `envelope`, `previous` (the previous amount, null when the Plan row did not exist), `amount`, and `status` (always `planned`).

```
{"ok":true,"year":2026,"month":11,"currency":"DKK",
 "rows":[{"envelope_id":1,"envelope":"Envelope
A","previous":null,"amount":4000,"status":"planned"}]}
```

Source: `api.php:set_plan_amounts`; `auth.php:FIN_API_TOKEN_WRITE_ACTIONS`.

9.14 Account balances API (external client)

```
GET api.php?action=accountBalances
```

Returns the current balance of every Account. The response carries `as_of` (the date the balances are effective as of), `currency` (the installation's Currency), and `accounts`.

Every Account carries:

- `id`
- `name`
- `currency` — the currency the reported balance is expressed in
- `balance` — the current balance; `null` when the Balance is unknown
- `balance_known` — `true` or `false`
- `status` — `active` or `archived`
- `last_observation_date` — the date of the latest Balance observation on or before `as_of`, or `null` when the Account has none

```
{"as_of":"2026-10-26","currency":"DKK",
 "accounts":[{"id":1,"name":"Account A","currency":"DKK","balance":1500,
              "balance_known":true,"status":"active",
              "last_observation_date":"2026-10-01"}]}
```

Rules:

- The reported balance is exactly the balance the Accounts screen shows for the same Account; the action does not calculate a second balance.
- An Account whose Balance is unknown reports `balance_known: false` and `balance: null`. It is never reported as zero.
- An Account in the Helper currency reports its balance in the Helper currency, and falls back to the Currency amount when that conversion is not available.
- Archived Accounts are included with `status: archived`; they are not filtered out.
- Account numbers are never reported.
- The action changes no data. It is a GET request; a POST to it needs the ordinary FIN session.

Source: `api.php:build_account_balances_response`; `accounting.php:get_accounts`.

9.15 Transfers API (external client)

```
GET api.php?action=transfers&from=YYYY-MM&to=YYYY-MM
```

Returns the Transfers of the inclusive calendar-month range. The response carries `from`, `to`, and `transfers`. Every transfer carries `kind`, which is either `envelope` or `account`.

An `envelope` transfer is money moved from one Envelope to another. It carries:

- `id`
- `date` — the date of the move
- `from_envelope`
- `to_envelope`
- `amount` — the moved amount, positive
- `note`
- `undone` — `true` when the move has been undone
- `payment_id`

An `account` transfer is money moved between two Accounts. It carries:

- `id`
- `date` — the date of the movement
- `from_account` — the source Account name
- `to_account` — the destination Account name
- `amount` — the moved amount, positive, in the Currency
- `currency` — the Currency the amount is in
- `note`
- `linked_to_statement_row` — `true` when the transfer is linked to an imported statement row, `false` otherwise
- `payment_id`


```
{"from":"2026-08","to":"2026-08",
 "transfers":[{"kind":"envelope","id":12,"date":"2026-08-05",
               "from_envelope":"Envelope A","to_envelope":"Envelope B",
               "amount":100,"note":"Move money","undone":false,
               "payment_id":"account:31+envelope:12"},
              {"kind":"account","id":31,"date":"2026-08-05",
               "from_account":"Account A","to_account":"Account B",
               "amount":100,"currency":"DKK","note":"Account transfer",
               "linked_to_statement_row":true,
               "payment_id":"account:31+envelope:12"}]}
```

Rules:

- The range may contain at most three calendar months. A longer range returns HTTP 400 with `Transfer range cannot exceed 3 months`.
- Selection follows the monthly report: the calendar month of the transfer date is authoritative, and future-dated transfers are excluded.
- An `envelope` transfer is the same move the monthly report shows as `Move money`, with the same date, amount, note, and Envelope names. An `account` transfer is the same movement the Accounts screen shows, with the same date, amount, note, and Account names.
- Expense and Refund Transactions are not transfers. Income events, Funding movements, Balance observations, and Balance adjustments are not transfers either.
- `payment_id` names the one real payment a row belongs to. One real payment can be recorded twice in FIN: once as an Envelope Transfer between two Envelopes with different default Accounts, and once as an Account Transfer between those Accounts. Both rows then carry the same `payment_id`, so a client counts that payment once instead of adding it up twice. A row that stands alone carries its own `payment_id`.
- Account numbers are never reported.
- The action changes no data. It is a GET request; a POST to it needs the ordinary FIN session.

Source: `api.php:build_transfers_response`; `api.php:report_balance_movement_rows`; `api.php:transfer_same_payment_pairs`.

---

10. Other essential recovery procedures

10.1 History archive (12-month retention)

```
php api.php archive-preview --keep-months=12     # only reports what would change
php api.php archive --keep-months=12             # executes; atomic transactional cleanup
```

Both share `$HOME/fin-backups` (or `FIN_BACKUP_DIR` / `--backup-dir=PATH`) for backups. `archive` performs:

1. SQLite integrity check.
2. Complete SQLite backup plus SHA-256 JSON manifest under `fin-archive-before-YYYY-MM-DD-TIMESTAMP-*.sqlite`.
3. Snapshot envelope balances.
4. One transactional cleanup: older movements are replaced with one `archive_rollup` brought-forward movement per Envelope at the first retained Month. Older months, allocations, expenses, account reminders, eligible transfer records, and superseded movements are removed. Transaction-linked Account events are detached before transaction detail is removed.
5. Verifies that envelope Balances are unchanged and foreign keys are intact.
6. Compacts the live database.

Cleanup aborts if any check fails; nothing is changed in the database until the backup has been verified.

`archive-preview` returns the same plan without changing data; always preview first.

Source: `api.php:archive_and_cleanup_database`; `api.php:run_cli` `archive-preview` / `archive`; `docs/architecture/CURRENT_STATE.md` "CLI surface" / "Running database retention".

10.2 Refresh development from production

```
FIN_PROD_SSH=user@production-host \
  ./scripts/refresh-dev-from-prod.sh
```

- The script requires:
- An ignored `config.local.php` in the development checkout.
- A valid external `master-pin-file` configuration in development (`master-pin-status --require`).
- A loopback `webauthn_origin` in development.
- A stopped development runtime (the script checks with `fuser` / `lsof`).
- It refuses if `FIN_PROD_SSH` is unset or malformed.
- It prompts before destroying the development database (or `FIN_REFRESH_CONFIRM=REPLACE_DEV` is required for non-interactive automation).

The remote production checkout runs `php api.php refresh-snapshot` (read-only, mode-`0600`), and the script:

- Validates the snapshot checksum.
- Migrates it to the development checkout's schema.
- Preserves the development `auth_principal` and `webauthn_credentials` (the production authentication state is intentionally replaced with the development equivalent).
- Backs up the previous development database under `fin-dev-before-prod-refresh-*.sqlite`.
- Atomically installs the candidate and prints `READY` only after final version / integrity / FK / auth / domain / permission / open-read verification.

Override optional paths with `FIN_PROD_ROOT`, `FIN_PROD_PHP`, `FIN_PROD_BACKUP_DIR`.

Source: `scripts/refresh-dev-from-prod.sh`; `api.php:create_refresh_snapshot`; `api.php:run_cli refresh-snapshot`.

10.3 Browser recovery

Symptom	Action
Lost / replaced browser	Sign in once with the Master PIN; the new browser is auto-enrolled (section 4.1).
Lost / cleared browser local storage (no `fin.deviceCredentialId`)	The sign-in screen shows the Master PIN form; sign in and the credential is re-enrolled.
Encryption wiped the WebAuthn private key	The browser no longer holds a credential; the Security shield on Month lists the old Label. Sign in once with the PIN, then revoke the unusable credential from the Security shield.
Shared machine	Always tap **Logout** (× button) when leaving FIN. The Master PIN still works on a shared machine, but a stolen credential on a shared machine lets the next user in.

Source: `app.js:logout`; `app.js:revokeDeviceCredential`; `index.html logoutBtn`.

10.4 Re-enrollment after origin change

See section 4.6. After moving FIN to a new host, change `webauthn_rp_id` and `webauthn_origin` in the resolved configuration (or override them in `config.local.php`) and sign in once with the Master PIN from each browser you intend to use.

Source: `auth.php:webauthn_server` / `auth.php:expected_webauthn_origin`; `config.php`.

Appendix A — Canonical command reference

Command	Source	Purpose
`php api.php version`	`api.php:run_cli`	Print `FIN Planner v<AppVersion> / database v<DbVersion>`.
`php api.php migrate`	`api.php:run_cli`	Apply unapplied migrations.
`php api.php set-master-pin [--stdin]`	`api.php:run_cli`	Configure or rotate the Master PIN (CLI only).
`php api.php master-pin-status [--require]`	`api.php:run_cli`	Print path + yes/no (exit 1 with `--require` when unconfigured).
`php api.php create-api-token`	`api.php:run_cli`	Create/rotate the API token and print it once.
`php api.php refresh-snapshot [--backup-dir=PATH]`	`api.php:run_cli`	One-way PROD → DEV snapshot (read-only).
`php api.php archive-preview [--keep-months=12]`	`api.php:run_cli`	Show what `archive` would remove.
`php api.php archive [--keep-months=12] [--backup-dir=PATH]`	`api.php:run_cli`	Backup + 12-month retention cleanup.

Command	Source	Purpose
`php api.php import-statements ACCOUNT_ID`	`api.php:run_cli`	Process dropped statement files for one Account.
`php api.php inspect-statement FILE`	`api.php:run_cli`	Print Import Profile header candidates for a file (no DB write).
`php api.php validate-import-profile PROFILE.json`	`api.php:run_cli`	Validate an Import Profile JSON.
`php api.php preview-import-profile PROFILE.json FILE`	`api.php:run_cli`	Show how a profile matches a file.
`php api.php save-import-profile PROFILE.json`	`api.php:run_cli`	Save an Import Profile.
`php api.php list-import-profiles`	`api.php:run_cli`	List Import Profiles.
`php api.php rename-import-profile ID NAME`	`api.php:run_cli`	Rename one Import Profile.
`php api.php enable-import-profile ID`	`api.php:run_cli`	Enable one Import Profile.
`php api.php disable-import-profile ID`	`api.php:run_cli`	Disable one Import Profile.
`php api.php delete-import-profile ID`	`api.php:run_cli`	Delete one Import Profile.

Command	Source	Purpose
`./scripts/update.sh`	`scripts/update.sh`	Production update.
`./scripts/refresh-dev-from-prod.sh`	`scripts/refresh-dev-from-prod.sh`	One-way PROD → DEV refresh.
`./scripts/test.sh [unit|integration|browser]`	`scripts/test.sh`	Test layers (synthetic fixtures only).
`./scripts/run-protected.sh`	`scripts/run-protected.sh`	Run the immutable protected-test harness. The harness location is installation-specific and is supplied through `FIN_PROTECTED_HARNESS_ROOT` or an ignored `scripts/run-protected.local.sh`; the wrapper fails with a clear message when neither is set.

Appendix B — Known gaps (no supported procedure today)

- Section 6.7 — FIN does not expose a general manual Account-to-Account Transfer UI; use statement reconciliation (§8) to record Account-to-Account Transfers.
- Section 8.2 — FIN ships the `import-statements` CLI and the drop-inbox processor, but no end-to-end operator runbook for the file-drop import path.